

Array List #1



Runtime Analysis Exercise

```
def count_evens(lst):  
    count = 0 #  $\Theta(1)$ 
```

$\Theta(n)$ $\left[\begin{array}{l} \text{for num in lst:} \\ \quad \text{if num \% 2 == 0:} \# \Theta(1) \\ \quad \quad \text{count += 1} \# \Theta(1) \end{array} \right.$

```
    return count #  $\Theta(1)$ 
```

$$T(n) = \cancel{\Theta(1)} + \Theta(n) = \boxed{\Theta(n)}$$

$$T(n) = \cancel{\Theta(1)} + \Theta(n^2) = \boxed{\Theta(n^2)}$$

$\Theta(n) \leftarrow \Theta(i)$

```
def unique_append(n):  
    result = [] #  $\Theta(1)$ 
```

$\Theta(n)$ $\left[\begin{array}{l} \text{for i in range(n):} \\ \quad \text{if i in result:} \\ \quad \quad \text{continue} \# \Theta(1) \\ \quad \text{result.append(i)} \end{array} \right.$

$\Theta(1) \leftarrow$

```
    return result #  $\Theta(1)$ 
```

```
def reduce_by_half(lst):  
    n = len(lst) #  $\Theta(1)$   
    count = 0 #  $\Theta(1)$ 
```

$\Theta(\log n)$ $\left[\begin{array}{l} \text{while n > 1:} \\ \quad \text{n = n // 2} \# \Theta(1) \\ \quad \text{count += 1} \# \Theta(1) \end{array} \right.$

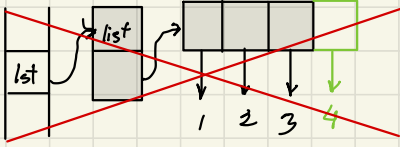
```
    return count #  $\Theta(1)$ 
```

$$T(n) = \cancel{\Theta(1)} + \Theta(\log n) = \boxed{\Theta(\log n)}$$

The Reality of Appending

`lst = [1, 2, 3]`

`lst.append(4)`



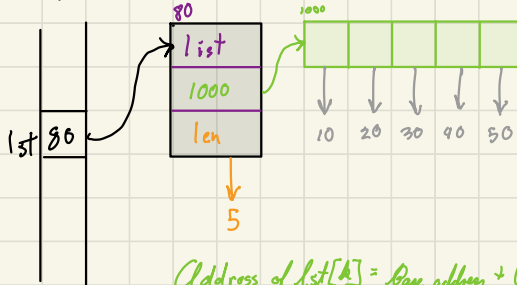
Process Name	Memory in Use
Python	0xfbc3b5ea - 0xfbc3c71a
	0x2e48a424 - 0x4e48a424
Zoom	0xd0c2f880 - 0xd4c2d133
Firefox	0x54356cfe - 0x75336afe
Slack	0x1dbbda19 - 0x1dbbcdee
Steam	0x8906e993 - 0x892c453e

Figure 2: Example of memory blocks being used by various processes.

Dynamic Arrays

- Array: fixed-size data structure
 - Stores a sequence of values
 - Stored contiguously (together in memory)
 - All elements must be of same size

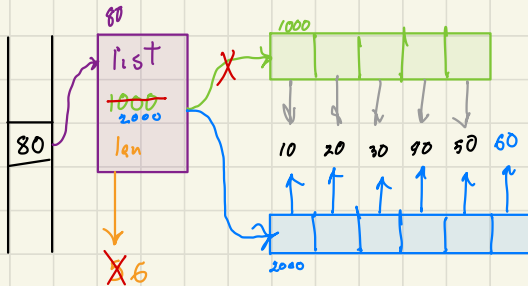
`lst = [10, 20, 30, 40, 50]`
`print(len(lst)) # 5`
`print(lst[4]) # 50`



Address of `lst[k]` = Base address + $(k * \text{size of an element})$
`lst[4]` = $0x4a003 + (4 * 8) = 0x4a023$

Dynamic Arrays / Array Lists

1. Preallocate extra memory : more memory than initially needed.
2. Use reserved space : append is $\Theta(1)$ so long as preallocated space is available.
3. Resize when full : If no space, create new array of larger size, copy elements, and index new elements.



append(60)

len 1	1
-------	---

len 2	1	2
-------	---	---

len 3	1	2	3
-------	---	---	---

⋮

len n	1	2	3	⋯	n
-------	---	---	---	---	---

$$\Theta(k) \rightarrow \Theta(n)$$

$$T(n) = 1 + 2 + 3 + 4 + \dots + n = \Theta(n^2)$$

size 1

1

(Best case) $\Omega(1)$: there is space available
(Worst case) $O(k)$: when reorganizing

size 2

1	2
---	---

$$T(n) = 1 + 2 + 3 + 4 + \dots + n = \Theta(n^2)$$

size 4

1	2	3	4
---	---	---	---

size 8

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

size n

1	2	3	4	5	6	7	8	...	n
---	---	---	---	---	---	---	---	-----	---

The average (or *amortized*) time per append operation is $\Theta(1)$

Amortized Analysis

$T_{\text{amort}}(n) = \text{the total cost of entire series of operations} / n$

$$T_{\text{insert, amort}}(n) = \Theta(n^2) / n = \Theta(n)$$

$$T_{\text{push, amort}}(n) = \Theta(n) / n = \Theta(1)$$